How to Run the Dynamic Firewall Project on a Mac

Step by step, with what every step does and what you should see

How it fits together

The project runs inside your Ubuntu VM (VirtualBox). You do not type project commands in the VM's own window. Instead you open **Terminal tabs on your Mac** and connect each one to the VM with **SSH**. VirtualBox forwards port 2222 on your Mac to port 22 (the SSH port) inside the VM. That is why every connection in this guide goes to `127.0.0.1` port `2222`.

Two kinds of prompt appear in this guide. The **Mac prompt** looks like `yourname@Your-MacBook ~ %`. The **VM prompt** looks like `pes2ug25am080@pes2ug25am080:~$`. Each step says which one to use. If your VM username is different, replace `pes2ug25am080` everywhere.

The four Terminal tabs you will use

Tab	Name	What runs in it	Stays busy?
1	Controller	ryu-manager: the SDN controller (the brain that tells the switch what to do)	Yes - leave it running
2	Network	Mininet: the virtual network. You type the demo commands here at the mininet> prompt	Yes
3	Log	tail -F on the security log: shows every alert, block and unblock as it happens	Yes
4	Switch	A live view of the firewall rules inside the switch (the proof the network changed)	Yes

Before you start, check you have

- A VirtualBox VM running Ubuntu 22.04 (you can start it in Step 1).
- A port-forwarding rule host 127.0.0.1 : 2222 -> guest 10.0.2.15 : 22 (Step 2 shows where).
- `dynamic-firewall.zip` downloaded on your Mac.
- Your VM username and password. Nothing is shown on screen while you type a password; that is normal.

The big picture in one paragraph

h1 runs a **log server**. h2 is a well-behaved, trusted client. h3 misbehaves. When h3 floods the server with messages, the log server tells the Ryu controller, and the controller installs a **drop rule** in the switch, so h3's traffic is discarded by the network itself. After h3 goes quiet, the rule is removed automatically. There are three kinds of rule: **block** (drop everything from one host), **deny** (drop one specific flow: source, destination, port) and **allow** (exempt a trusted host).

Part 1 - One-time setup

Do these steps once. After they are done, skip to Part 2 every time you want to run the project.

STEP 1 Start the VM

VirtualBox on the Mac

Do this: Open VirtualBox, select your VM and click **Start**. Wait until the Ubuntu desktop or login screen appears. You do not need to log in there. You can minimise the window.

What it does: Boots the Linux machine that runs Mininet, Open vSwitch and Ryu.

You should see: The Ubuntu screen. Leave the VM running for the rest of the guide.

STEP 2 Check the SSH port forward

VirtualBox on the Mac

Do this: VM **Settings > Network > Adapter 1 > Advanced > Port Forwarding**. You need one rule: Protocol TCP, Host IP 127.0.0.1, Host Port 2222, Guest IP 10.0.2.15, Guest Port 22.

What it does: This rule is the bridge between your Mac and the VM's SSH server. Without it, SSH cannot reach the VM.

You should see: The rule listed. If the fields are greyed out, shut the VM down, edit the rule, and start it again.

Note: Only needed once; the rule is saved with the VM.

STEP 3 Test the SSH connection

On the Mac (Terminal)

```
ssh -p 2222 pes2ug25am080@127.0.0.1
```

What it does: Opens a text session inside the VM. The first time, SSH asks you to confirm the VM's fingerprint: type `yes`. Then enter the VM password.

You should see: The VM prompt `pes2ug25am080@pes2ug25am080:~$`. Type `exit` to come back to the Mac prompt.

STEP 4 Copy the project to the VM

On the Mac (Terminal)

```
cd ~/Downloads
scp -P 2222 dynamic-firewall.zip pes2ug25am080@127.0.0.1:~/
```

What it does: Goes to the folder where the zip was saved (change it if yours is elsewhere), then copies the file into the VM's home folder over the same forwarded port. Note the **capital P**: scp uses -P for the port, ssh uses -p.

You should see: A progress line ending in 100%.

STEP 5 Unpack the project

On the VM (any SSH tab)

```
ssh -p 2222 pes2ug25am080@127.0.0.1
cd ~
rm -rf dynamic-firewall __MACOSX # only if an older copy exists
unzip dynamic-firewall.zip
cd dynamic-firewall
ls
```

What it does: Connects, goes to your home folder, deletes an older copy if there is one, and unzips the new project into `~/dynamic-firewall`. The __MACOSX folder is harmless Mac metadata inside the zip.

You should see: A listing with: app config controller docs experiments logs README.md requirements.txt results scripts tests topology.

Note: The rm -rf line deletes the old project folder including its logs. Copy any screenshots or logs you want to keep first.

STEP 6 Install the software

On the VM (any SSH tab)

```
./scripts/install_deps.sh
source ~/.bashrc
```

What it does: Installs **Mininet** (the virtual network), **Open vSwitch** (the software switch) and **iperf** (bandwidth tests). Turns off the openvswitch-testcontroller service, which would otherwise hold port 6653 and block Ryu. Installs **Ryu** from GitHub with compatible eventlet and dnspython (plain *pip install ryu* crashes on Python 3.10). Installs matplotlib for the charts. Adds `~/.local/bin` to your PATH so `ryu-manager` is found. It asks for your sudo password once and takes about 5-10 minutes.

You should see: The last lines say `ryu-manager 4.34` and `Done.`

Note: If you get 'Permission denied' on the first line, run `bash scripts/install_deps.sh` instead.

STEP 7 Run the automatic tests

On the VM (any SSH tab)

```
./scripts/run_tests.sh
```

What it does: Runs 43 tests without Mininet: the controller's rule engine (it builds real OpenFlow messages against a pretend switch), the log server's policy logic, and an end-to-end run on localhost with the real Ryu controller. It proves the software works before you touch the virtual network.

You should see: About a minute of output, ending with `ALL TESTS PASSED`.

Note: Ryu must not be running while the tests run (port 8080 must be free).

STEP 8 Check that Mininet works

On the VM (any SSH tab)

```
sudo mn --test pingall
sudo mn -c
```

What it does: Builds a tiny two-host network with one switch, pings between the hosts, and tears it down. The second line cleans up anything left over. If this works, Mininet and Open vSwitch are fine.

You should see: `Results: 0% dropped (2/2 received)`.

Note: If it says a controller is already running on port 6653, run `sudo pkill -x ovs-testcontrol` and try again.

Part 2 - Start the system (every time)

Follow this order exactly. The order matters: **sudo mn -c kills Ryu**, so clean up first and start Ryu second.

STEP 9 Open four Terminal tabs and connect each to the VM

On the Mac (Terminal)

Do this: With the VM running, press **Cmd+T** three times so you have four tabs. In **each** tab run the two lines below. Name the tabs in your head: 1 Controller, 2 Network, 3 Log, 4 Switch.

```
ssh -p 2222 pes2ug25am080@127.0.0.1
cd ~/dynamic-firewall
```

What it does: Creates four independent sessions on the same VM, one for each program that must stay running at the same time.

You should see: All four tabs show `pes2ug25am080@pes2ug25am080:~/dynamic-firewall$`.

STEP 10 Clean up leftovers

VM - Tab 2 (Network)

```
sudo mn -c
rm -f logs/security_events.log logs/server_stdout.txt
```

What it does: Removes any virtual hosts and switches left by an earlier run, and deletes the old logs so this run's log starts empty.

You should see: Lines starting with *** ending in `*** Cleanup complete.`

Note: Do this before starting Ryu. mn -c kills any running ryu-manager.

STEP 11 Start the controller

VM - Tab 1 (Controller)

```
ryu-manager controller/firewall_controller.py
```

What it does: Starts the SDN controller. It listens on port 6653 for the switch (OpenFlow) and on port 8080 for the REST commands that the log server sends. It holds the rule engine (allow, deny, block) and polls the switch's statistics every second.

You should see: `wsgi starting up on http://0.0.0.0:8080`, then it goes quiet. Leave it running.

STEP 12 Start the virtual network

VM - Tab 2 (Network)

```
sudo python3 topology/firewall_topo.py
```

What it does: Mininet creates three hosts (h1 = 10.0.0.1 log server, h2 = 10.0.0.2 well-behaved and trusted, h3 = 10.0.0.3 the flooder), one Open vSwitch switch s1, and a NAT node at 10.0.0.254 that lets the hosts reach the controller's REST API. It connects s1 to Ryu.

You should see: The `mininet>` prompt in Tab 2, and `Switch 1 connected` in Tab 1.

STEP 13 Check the network and the controller

VM - Tab 2 (Network) (mininet> prompt)

```
pingall
h1 curl http://10.0.0.254:8080/firewall/status
```

What it does: `pingall` makes every host ping every other host. The second line runs curl on h1 (the `h1` prefix means 'run this on host h1') to ask the controller for its status. That proves h1 can reach the controller.

You should see: `0% dropped (12/12 received)` and `{"blocked_ips": [], "rules": 0}`.

Note: If pingall shows X, Ryu is not running or the switch has not connected yet. Look at Tab 1, wait about 10 seconds and try again.

STEP 14 Start the log server on h1

VM - Tab 2 (Network) (mininet> prompt)

```
h1 python3 app/log_server.py --controller 10.0.0.254:8080 > logs/server_stdout.txt 2>&1 &
```

What it does: Starts the TCP log server on h1. It listens on port 9000 (the logging service, rate-limited) and port 9001 (the admin service, allow-list only). The part after the command sends its screen output to a file and the final `&` runs it in the background so the prompt comes back. Within about 2 seconds it asks the controller to install an ALLOW rule for the trusted host h2.

You should see: The prompt returns immediately with no message.

STEP 15 Watch the security log

VM - Tab 3 (Log)

```
tail -F logs/security_events.log
```

What it does: Prints each new line of the log as it is written. Every connection, alert, block and unblock appears here.

You should see: Lines such as `Log server starting`, `Listening on 0.0.0.0:9000`, `Listening on 0.0.0.0:9001` and `ALLOW rule for trusted source 10.0.0.2 installed on the controller`.

STEP 16 Watch the firewall rules inside the switch

VM - Tab 4 (Switch)

```
sudo watch -n1 "ovs-ofctl -O OpenFlow13 dump-flows s1 | grep -E 'actions=drop|priority=300'"
```

What it does: Every second, asks Open vSwitch for its flow table and shows only the firewall rules. Priority 300 is an allow rule, priority 200 is a deny rule (one flow dropped) and priority 100 is a block rule (a whole host dropped). This is the proof that the *network itself* changed, not just the application.

You should see: One line like `priority=300,ip,nw_src=10.0.0.2 actions=goto_table:1` (the allow rule for h2). Press Ctrl+C to stop the watch when you are finished.

Part 3 - The demo scenarios

Type these in **Tab 2** at the `mininet>` prompt while you watch Tabs 3 and 4. Wait about 20 seconds between scenarios so earlier blocks expire. You can check with `h1 curl http://10.0.0.254:8080/firewall/status`: it should list no blocked IPs.

SCENARIO A A flood is blocked, then recovers (BLOCK rule)

Tab 2 at the mininet> prompt

```
h2 python3 app/client.py --server 10.0.0.1 --mode normal --count 6
h3 python3 app/client.py --server 10.0.0.1 --mode flood --count 40
h3 ping -c 3 10.0.0.1
```

What each command does: Line 1: h2 sends 6 messages, 2 seconds apart. Line 2: h3 sends a message every 0.1 seconds. The log server counts messages per source in a 5-second window and allows 8, so the 9th is too many. Line 3: tests whether h3 can still reach h1 (run it right after line 2).

You should see: Line 1: six ACK replies. Line 2: ACK eight times, then `BLOCKED`, then a 'connection error ... timed out'. Tab 3: `ALERT ... requesting SDN block`, then `BLOCKED by SDN controller`, and about 16 seconds later `UNBLOCKED after ... quiet`. Tab 4: `priority=100,ip,nw_src=10.0.0.3 actions=drop` appears, then disappears. Line 3: 100% packet loss while blocked, normal replies after the unblock.

What it proves: Something the application saw (too many messages) changed the network (a drop rule in the switch), and the network recovered by itself.

SCENARIO B Deny one flow only (DENY rule)

Tab 2 at the mininet> prompt

```
h2 python3 app/client.py --server 10.0.0.1 --port 9001 --count 3 --delay 0.5
h3 python3 app/client.py --server 10.0.0.1 --port 9001 --count 3
h3 python3 app/client.py --server 10.0.0.1 --port 9000 --count 3 --delay 0.5
```

What each command does: Port 9001 is the 'admin' service and only h2 is on its allow-list. Line 1: h2 uses it (allowed). Line 2: h3 tries it (not allowed). Line 3: h3 uses the normal logging port 9000.

You should see: Line 1: three ACK. Line 2: one reply, `DENIED`. Tab 3: `ALERT: POLICY VIOLATION 10.0.0.3 -> 10.0.0.1:9001` and `DENY ... installed for 30s`. Tab 4: a priority=200 line `tcp,nw_src=10.0.0.3,nw_dst=10.0.0.1,tp_dst=9001 actions=drop` for 30 seconds. Line 3: still three ACK.

What it proves: Traffic is classified by source, destination and port. The response is different from a block: only h3's flow to port 9001 is dropped and everything else from h3 keeps working.

Note: If you repeat line 2 within 30 seconds it waits about 5 seconds and then says it could not connect: the switch is now dropping h3's connection attempts to that port.

SCENARIO C A trusted host is exempt (ALLOW rule)

Tab 2 at the mininet> prompt

```
h2 python3 app/client.py --server 10.0.0.1 --mode flood --count 40
```

What each command does: h2 sends the same fast flood that got h3 blocked in Scenario A.

You should see: Every reply is ACK and h2 is never blocked. Tab 3 shows `class=trusted` on h2's connection. Tab 4 still shows the priority=300 allow line.

What it proves: An allow rule (priority 300) overrides drop rules and the rate limit. Allow is an operator decision, set in `config/policy.json` as trusted_sources.

SCENARIO D Controller failure and recovery

Tab 2 at the mininet> prompt

```
# 1. Tab 1: press Ctrl+C to stop Ryu
h3 python3 app/client.py --server 10.0.0.1 --mode flood --count 40
# 2. Tab 1: ryu-manager controller/firewall_controller.py
# 3. wait about 15 seconds, then in Tab 2:
pingall
```

What each command does: Stops the controller, floods while it is down, restarts it, then checks the network.

You should see: With Ryu down, every reply stays ACK (nothing can be blocked) and Tab 3 logs **one** warning that the controller is unreachable, not one per message. After the restart Tab 3 says the controller is reachable again and re-installs the ALLOW rule; Tab 4 shows the priority=300 line again; pingall reports 0% dropped.

What it proves: The log server keeps working without the controller, and when the controller returns the firewall rules are restored automatically.

Note: Wait at least 20 seconds after Scenario A or B first, so h3 is not blocked when you start.

SCENARIO E Detection from the switch's own statistics (optional)

Tab 2 at the mininet> prompt

```
h1 pkill -f "[l]og_server.py"
h1 python3 app/log_server.py --controller 10.0.0.254:8080 --threshold 100000
--bw-threshold-mbps 3 > logs/server_stdout.txt 2>&1 &
h3 python3 app/client.py --server 10.0.0.1 --mode flood --rate 0 --size 1400 --connections 2
--duration 10 --persist --quiet
```

What each command does: Line 1 stops the log server. Line 2 restarts it with message counting effectively switched off (limit 100000) and a rule to block any source the *switch* measures above 3 Mbit/s. Line 3 sends 1400-byte messages as fast as possible on two connections for 10 seconds.

You should see: Tab 3: `ALERT: 10.0.0.3 measured ... Mbps by the switch` followed by the block, and a drop rule in Tab 4.

What it proves: Detection can come from network measurements (OpenFlow flow statistics polled by the controller), not only from the application. Restart the normal log server (Step 14) afterwards.

Screenshots to take for your report

- Scenario A: the h3 output (ACK x8, BLOCKED, timeout) next to the Tab 4 drop rule.
- Tab 3 showing ALERT, BLOCKED by SDN controller and UNBLOCKED lines with their timestamps (these give your response and recovery times).
- Scenario B: the DENIED reply and the priority=200 rule, then h3 still getting ACK on port 9000.
- Scenario C: h2's flood with all ACK and the priority=300 allow rule.
- Optionally the output of `curl -s http://127.0.0.1:8080/firewall/rules` and `.../firewall/stats` run on the VM: they show packet counters for the drop rules.

Part 4 - The experiments (baseline vs dynamic firewall)

This produces the numbers for the performance part of the report: what a normal user (h2) experiences while h3 floods, with and without the dynamic firewall.

STEP 17 Get ready

Tabs 2, 3 and 4

Do this: In Tab 2 type `exit` at the mininet> prompt. In Tabs 3 and 4 press Ctrl+C. Leave Ryu running in Tab 1.

What it does: The experiment script builds its own network, so no other Mininet session may be open. It needs Ryu running.

Note: Do **not** run sudo mn -c now. It would kill Ryu.

STEP 18 Quick trial run (about 2-3 minutes)

VM - Tab 2 (Network)

```
sudo python3 experiments/run_experiments.py --quick
```

What it does: Runs one short trial for every combination of **condition** and **flood intensity**. The conditions are **idle** (no flood), **static** (the baseline: the flood runs but the log server only monitors, so the network never reacts) and **dynamic** (the SDN block is active). The intensities are low (1 connection, 20 messages/s), medium (1 connection, as fast as possible) and high (4 connections, saturating the 10 Mbit/s server link). During each trial h2 measures its own throughput (iperf) and latency (ping) at the same time as the flood.

You should see: One progress line per trial, for example `[ 3/7] static low run1 iperf=... Mbit/s ping avg=... ms`, then `Done.` It writes `results/results.csv`.

STEP 19 Full run (about 10 minutes)

VM - Tab 2 (Network)

```
rm -f results/results.csv
sudo python3 experiments/run_experiments.py
```

What it does: Same as Step 18 with 20-second trials, repeated 3 times for every combination (21 trials), so the averages are meaningful. The first line deletes the quick run's file, because new results are appended to the existing CSV.

You should see: 21 progress lines and `Done.`

STEP 20 Make the tables and charts

VM - Tab 2 (Network)

```
python3 experiments/analyze.py
```

What it does: Reads the CSV, prints a results table (mean and spread over the runs) and writes `results/summary.md` plus three charts: h2 throughput, h2 latency and attacker traffic accepted.

You should see: Tables on screen and lines such as `wrote results/h2_throughput.png`.

STEP 21 Copy the results to your Mac

On the Mac (Terminal tab, not connected by SSH)

```
mkdir -p ~/Desktop/results
scp -P 2222 "pes2ug25am080@127.0.0.1:~/dynamic-firewall/results/*" ~/Desktop/results/
```

What it does: Copies summary.md, results.csv and the PNG charts to a folder on your Desktop. scp skips the raw folder with a harmless message.

You should see: The files in `~/Desktop/results`.

How to read the results

- **Idle** is what h2 gets on a quiet network. It is your reference line.
- **Static** should get worse as the intensity rises: lower h2 throughput, higher latency, because the flood is competing for the same 10 Mbit/s link.
- **Dynamic** should stay close to idle, because the flood is cut off within about a second and stays cut off while blocked. The 'packets dropped by the switch' column shows the firewall doing the work.
- At **low** intensity the flood is too light to congest the link, so static and dynamic can look similar. That is expected and worth saying in the report: the firewall still catches the policy violation, but there is little network impact to save.
- Report whatever you measure, including surprises. Send me summary.md and I will help you interpret it.

Part 5 - Shutting down

STEP 22 Stop everything in order

Tabs 2, 3, 4, then 1

```
# Tab 2: exit
# Tabs 3 and 4: Ctrl+C
# Tab 1: Ctrl+C
# Tab 2 (last): sudo mn -c
```

What it does: exit stops Mininet cleanly. Ctrl+C stops the log viewer, the rule watcher and then Ryu. The final sudo mn -c removes anything left behind. It goes last because it also kills Ryu.

STEP 23 Save your work

On the Mac (Terminal)

```
scp -P 2222 -r pes2ug25am080@127.0.0.1:~/dynamic-firewall/logs ~/Desktop/logs-backup
```

What it does: Copies the logs from the last run to your Mac. Also push the project to GitHub (the commands are in README.md, section 'Publishing to GitHub'); the rubric requires a GitHub repository.

STEP 24 Turn the VM off

On the VM (any SSH tab)

```
sudo poweroff
```

What it does: Shuts the VM down cleanly. Alternatively use VirtualBox: Machine > ACPI Shutdown.

Troubleshooting

What you see	Likely cause	Fix
ssh: Connection reset by peer, or Connection refused	VM not fully booted, SSH server not running, or the port-forward rule is missing	Wait a minute after starting the VM. Check the rule (Step 2). In the VM's own window run sudo systemctl start ssh.
Host key verification failed	The VM was reinstalled, so its fingerprint changed	On the Mac: ssh-keygen -R "[127.0.0.1]:2222", then connect again.
ryu-manager: command not found	~/.local/bin is not on PATH in this tab	Run source ~/.bashrc, or open a new tab, or export PATH=$PATH:$HOME/.local/bin
Ryu crashes at start with is_timeout or ALREADY_HANDLED	Wrong Ryu / eventlet versions	Run ./scripts/install_deps.sh again (Step 6).
Please shut down the controller which is running on port 6653	The test-controller service, or another Ryu, is using the port	sudo pkill -x ovs-testcontrol. If it is another Ryu, press Ctrl+C in that tab.
pingall shows X	Ryu is not running or the switch has not connected yet	Start Ryu (Step 11), wait about 10 seconds, try again. sudo ovs-vsctl show should say is_connected: true.
Ryu stopped on its own	sudo mn -c was run while Ryu was running	Restart Ryu. Always run mn -c before starting Ryu.
h1 curl hangs or is refused	Ryu is not running, or the topology was not started with firewall_topo.py (no NAT node)	Check Tab 1, restart the topology with Step 12.
cannot listen on port 9000	A log server is already running	h1 pkill -f "[l]og_server.py" at the mininet> prompt, then start it again.
The flood never shows BLOCKED	The log server is not running, h3 stayed under the limit, or the controller is down	Check Tab 3 for the log server lines and Tab 1 for Ryu. Redo Step 14.
The rule watcher in Tab 4 is blank	No rule exists right now (drop rules last 15 s or 30 s)	Run the flood and look straight away. The output uses nw_src and tp_dst, not ipv4_src.
Experiments: Cannot reach the Ryu controller	Ryu is not running	Start Ryu in Tab 1 first (Step 11).
Experiments: h1 cannot reach the REST API	A stale network from an earlier run	Close any mininet> session with exit, then run the experiment again. Do not use mn -c while Ryu runs.

One-page cheat sheet

After the one-time setup, this is everything you type. Replace the username if yours differs.

Every tab (Mac Terminal)

```
ssh -p 2222 pes2ug25am080@127.0.0.1
cd ~/dynamic-firewall
```

Start-up, in this order

```
# Tab 2
sudo mn -c
rm -f logs/security_events.log logs/server_stdout.txt
# Tab 1
ryu-manager controller/firewall_controller.py
# Tab 2
sudo python3 topology/firewall_topo.py
pingall
h1 curl http://10.0.0.254:8080/firewall/status
h1 python3 app/log_server.py --controller 10.0.0.254:8080 > logs/server_stdout.txt 2>&1 &
# Tab 3
tail -F logs/security_events.log
# Tab 4
sudo watch -n1 "ovs-ofctl -O OpenFlow13 dump-flows s1 | grep -E 'actions=drop|priority=300'"
```

Demo (Tab 2, mininet> prompt)

```
# A: normal client, all ACK
h2 python3 app/client.py --server 10.0.0.1 --mode normal --count 6
# A: flood, gets blocked
h3 python3 app/client.py --server 10.0.0.1 --mode flood --count 40
# B: allowed on the admin port
h2 python3 app/client.py --server 10.0.0.1 --port 9001 --count 3 --delay 0.5
# B: denied on the admin port
h3 python3 app/client.py --server 10.0.0.1 --port 9001 --count 3
# C: trusted host is never blocked
h2 python3 app/client.py --server 10.0.0.1 --mode flood --count 40
```

Experiments

```
# Tab 2: exit the mininet> prompt first; keep Ryu running in Tab 1
rm -f results/results.csv
# smoke test, 2-3 minutes
sudo python3 experiments/run_experiments.py --quick
# full run, about 10 minutes
sudo python3 experiments/run_experiments.py
python3 experiments/analyze.py
```

Shut down

```
# Tab 2: exit | Tabs 3, 4, 1: Ctrl+C | Tab 2: sudo mn -c | VM off: sudo poweroff
```

Three rules that prevent most problems: (1) run sudo mn -c before starting Ryu, never while it is running; (2) start Ryu before the topology; (3) the experiment script needs Ryu running and no mininet> session open.